Lekcja 07: Logika i pętle

`#lekcja` `#datascience` `#python` `#kontrola-przeplywu` `#petla` `#logika`

W tej lekcji poznasz podstawowe mechanizmy kontroli przepływu w Pythonie: instrukcje warunkowe (`if/elif/else`) oraz pętle (`while` i `for`). Nauczysz się, jak automatyzować powtarzalne zadania i podejmować decyzje na podstawie danych – kluczowe umiejętności w Data Science. Dowiesz się również, jak pisać efektywny kod, który zużywa mniej zasobów obliczeniowych.

---

1. Instrukcje warunkowe (if/elif/else)

Definition

Instrukcje warunkowe pozwalają wykonywać różne fragmenty kodu w zależności od spełnienia określonych warunków logicznych. W Data Science są niezbędne do filtrowania danych, klasyfikacji próbek i podejmowania decyzji na podstawie wartości zmiennych.

Wprowadzenie

Instrukcje warunkowe to fundament logiki programowania. Pozwalają tworzyć programy, które reagują na różne sytuacje – od prostego sprawdzania wartości po złożoną klasyfikację danych. W Data Science używamy ich do:

- Filtrowania obserwacji (np. wybierz tylko wiersze, gdzie wiek > 18)
- Kategoryzacji danych (np. BMI → "niedowaga", "norma", "nadwaga")
- Walidacji danych wejściowych
- Implementacji reguł biznesowych

Przykład 1: Podstawowa instrukcja if/else - Klasyfikacja temperatury

```
# Analiza temperatury dla zbioru danych pogodowych
# Podstawowa klasyfikacja: zimno/ciepło

temperatura_celsius = 22.5

if temperatura_celsius >= 20:
    kategoria = "ciepło"
```

```
    print(f"Temperatura {temperatura_celsius}°C to {kategoria}")
else:
    kategoria = "zimno"
    print(f"Temperatura {temperatura_celsius}°C to {kategoria}")

# Przykład z danymi rzeczywistymi - analiza pojedynczej obserwacji
temperatura_pomiar = 15.3

if temperatura_pomiar < 0:
    print(f"Poniżej zera: {temperatura_pomiar}°C - ryzyko oblodzenia")
elif temperatura_pomiar < 10:
    print(f"Chłodno: {temperatura_pomiar}°C - ogrzewanie zalecane")
elif temperatura_pomiar < 20:
    print(f"Umiarkowanie: {temperatura_pomiar}°C - komfort termiczny")
else:
    print(f"Ciepło: {temperatura_pomiar}°C - możliwa klimatyzacja")
```

Note

W Pythonie bloki kodu definiowane są przez wcięcia (indentację), nie przez klamry. Standardowo używa się 4 spacji.

Przykład 2: Złożone warunki - Kategoryzacja BMI

```
# Body Mass Index (BMI) - klasyfikacja zdrowotna
# BMI = waga (kg) / wzrost^2 (m^2)

waga_kg = 75
wzrost_m = 1.78

# Obliczenie BMI
bmi = waga_kg / (wzrost_m ** 2)
print(f"BMI: {bmi:.2f}")

# Wielopoziomowa klasyfikacja z użyciem elif
if bmi < 16:
    kategoria = "wygłodzenie"
    zalecenie = "natychmiastowa konsultacja lekarska"
elif bmi < 18.5:
    kategoria = "niedowaga"
    zalecenie = "zwiększenie kaloryczności diety"
elif bmi < 25:
    kategoria = "waga prawidłowa"
    zalecenie = "utrzymanie obecnego stylu życia"
```

```python
elif bmi < 30:
    kategoria = "nadwaga"
    zalecenie = "redukcja kalorii i aktywność fizyczna"
elif bmi < 35:
    kategoria = "otyłość I stopnia"
    zalecenie = "dieta pod kontrolą dietetyka"
else:
    kategoria = "otyłość II stopnia lub wyższa"
    zalecenie = "konsultacja z lekarzem specjalistą"

print(f"Kategoria: {kategoria}")
print(f"Zalecenie: {zalecenie}")

# Operator logiczny: łączenie warunków
# Sprawdzamy, czy BMI jest w zakresie normalnym I wzrost jest prawidłowy
if 18.5 <= bmi < 25 and wzrost_m > 1.5:
    print("✓ Parametry zdrowotne w normie")
else:
    print("⚠ Zalecane badania kontrolne")
```

Tip

Używaj `elif` zamiast wielu zagnieżdżonych `if` - kod jest bardziej czytelny i wydajniejszy (Python sprawdza warunki sekwencyjnie i zatrzymuje się na pierwszym prawdziwym).

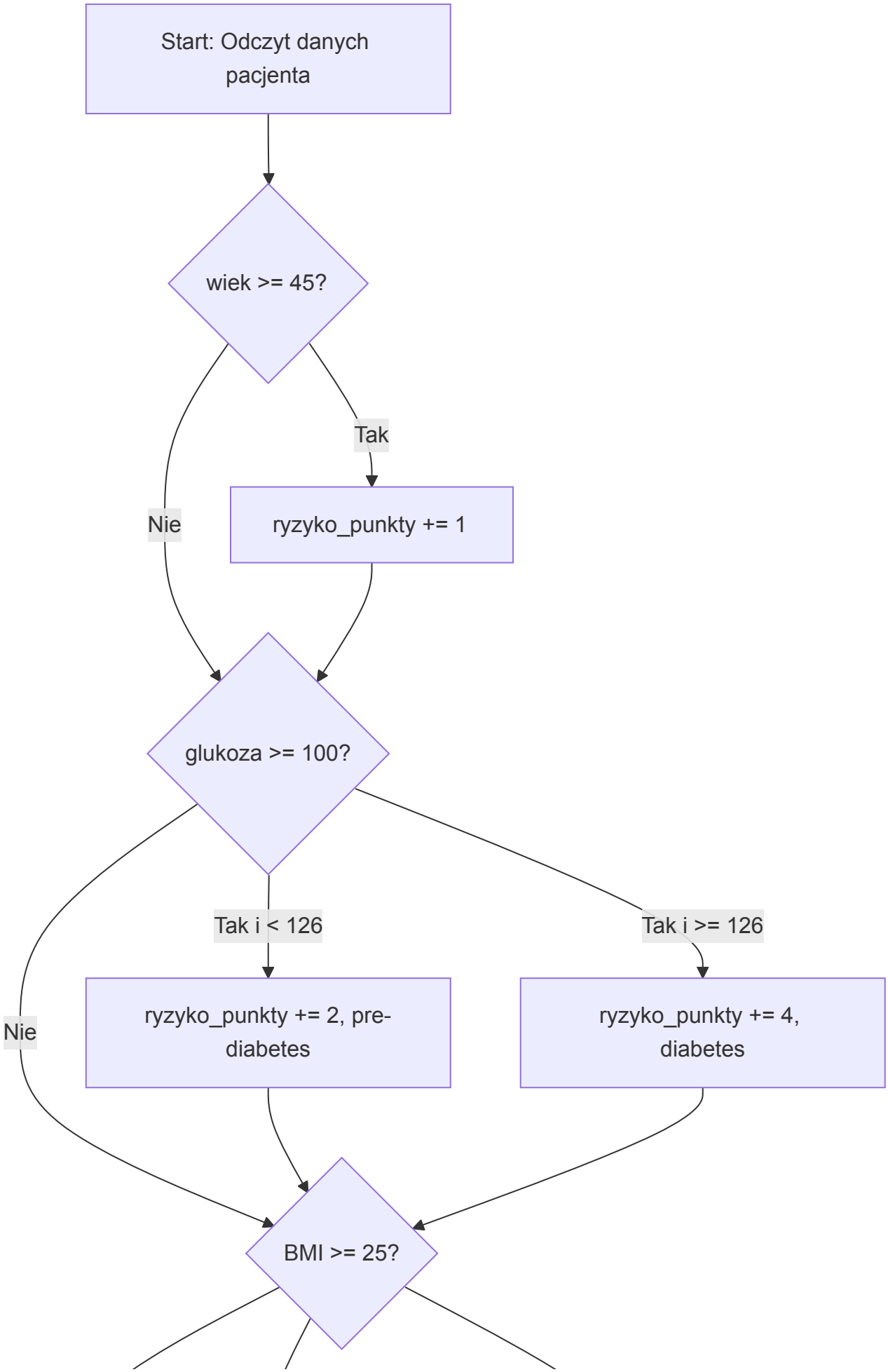
Przykład 3: Zaawansowane warunki - Filtrowanie danych pacjentów

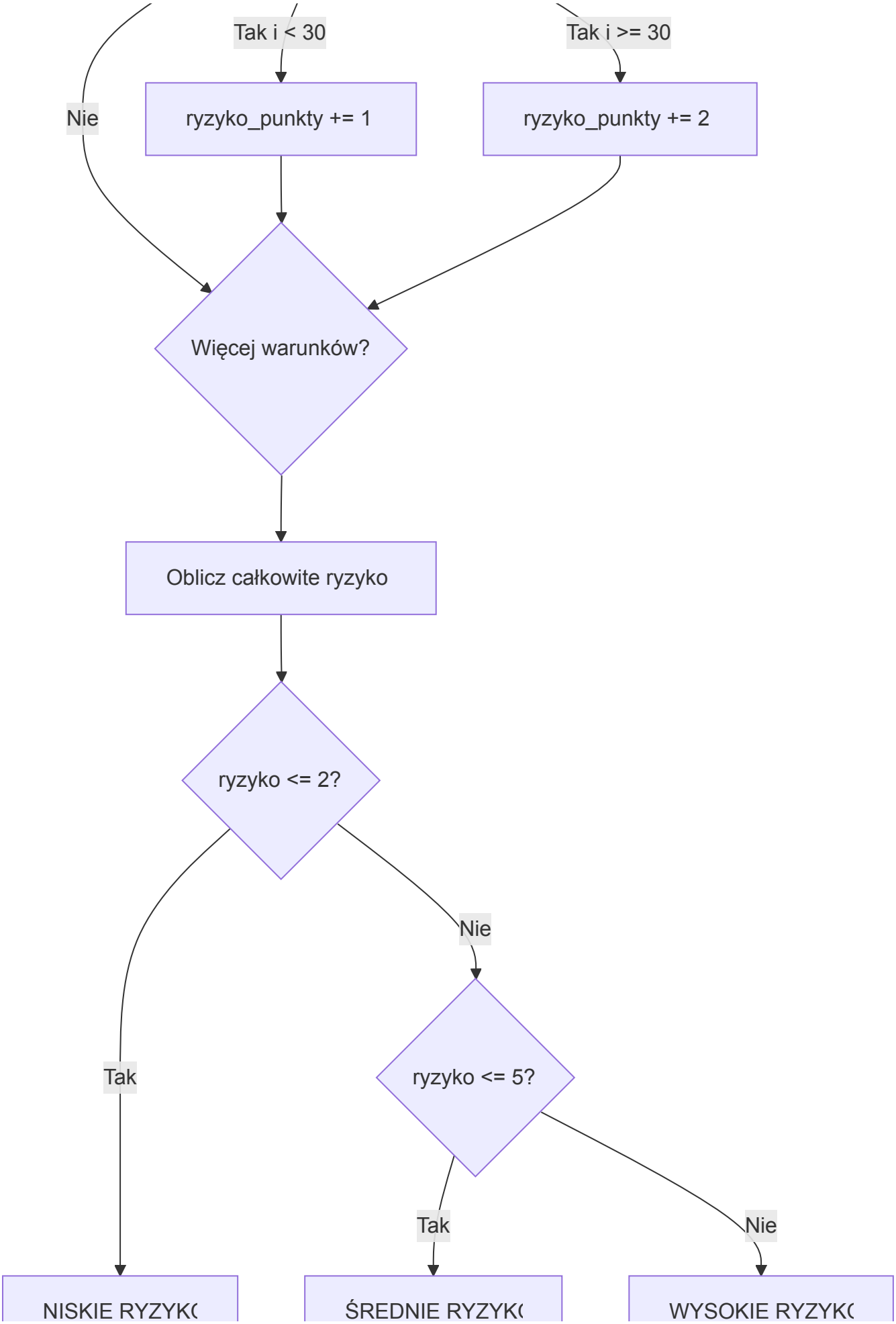
```python
# Symulacja prostego systemu diagnostycznego
# Analiza ryzyka cukrzycy na podstawie parametrów zdrowotnych

# Dane pacjenta (symulacja pojedynczej obserwacji z datasetu medycznego)
wiek = 45
glukoza_na_czczo = 110  # mg/dL
bmi = 28.5
cisnienie_skurczowe = 135  # mmHg
wywiad_rodzinny_cukrzyca = True

# Złożona ocena ryzyka z użyciem operatorów logicznych (and, or, not)
ryzyko_punkty = 0

# Wiek
if wiek >= 45:
    ryzyko_punkty += 1
```

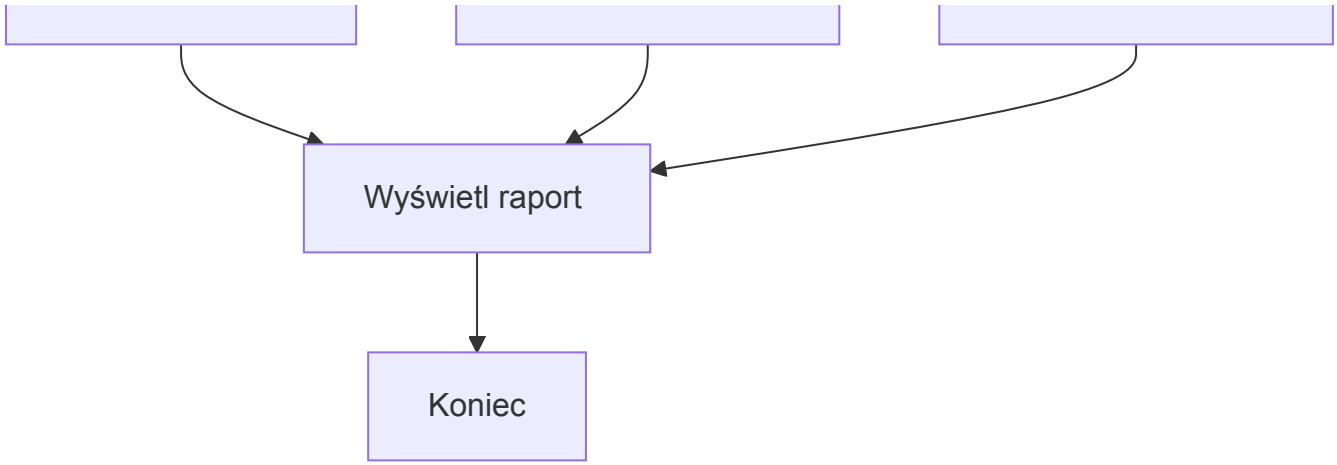
```python
# Glukoza
if glukoza_na_czczo >= 100 and glukoza_na_czczo < 126:
    ryzyko_punkty += 2  # Pre-diabetes
    status_glukozy = "podwyższona (pre-diabetes)"
elif glukoza_na_czczo >= 126:
    ryzyko_punkty += 4  # Diabetes
    status_glukozy = "bardzo wysoka (diabetes)"
else:
    status_glukozy = "prawidłowa"

# BMI
if bmi >= 25 and bmi < 30:
    ryzyko_punkty += 1  # Nadwaga
elif bmi >= 30:
    ryzyko_punkty += 2  # Otyłość

# Ciśnienie
if cisnienie_skurczowe >= 130:
    ryzyko_punkty += 1  # Nadciśnienie

# Wywiad rodzinny
if wywiad_rodzinny_cukrzyca:
    ryzyko_punkty += 2

# Klasyfikacja końcowa
print("=== RAPORT OCENY RYZYKA CUKRZYCY ===")
print(f"Wiek: {wiek} lat")
print(f"Glukoza na czczo: {glukoza_na_czczo} mg/dL ({status_glukozy})")
print(f"BMI: {bmi:.1f}")
print(f"Ciśnienie skurczowe: {cisnienie_skurczowe} mmHg")
print(f"Wywiad rodzinny: {'Tak' if wywiad_rodzinny_cukrzyca else 'Nie'}")
print(f"\nPunkty ryzyka: {ryzyko_punkty}")

if ryzyko_punkty <= 2:
    poziom_ryzyka = "NISKIE"
    rekomendacja = "Kontrola raz w roku"
elif ryzyko_punkty <= 5:
    poziom_ryzyka = "ŚREDNIE"
    rekomendacja = "Kontrola co 6 miesięcy, modyfikacja stylu życia"
else:
    poziom_ryzyka = "WYSOKIE"
    rekomendacja = "Pilna konsultacja diabetologiczna, badania szczegółowe"

print(f"\nPoziom ryzyka: {poziom_ryzyka}")
print(f"Rekomendacja: {rekomendacja}")
```

> ⚠ **Warning**
>
> Uważaj na kolejność warunków w `if/elif/else`! Python sprawdza je od góry do dołu i wykonuje tylko pierwszy prawdziwy blok. Np. jeśli wpiszesz `if x > 0:` przed `if x > 10:`, to dla x=15 wykona się tylko pierwszy warunek.

Wizualizacja przepływu instrukcji warunkowej


Start: Odczyt danych pacjenta
wiek >= 45?
Tak
Nie
ryzyko_punkty += 1
glukoza >= 100?
Tak i < 126
Tak i >= 126
Nie
ryzyko_punkty += 2, pre-diabetes
ryzyko_punkty += 4, diabetes
BMI >= 25?


Tak i < 30
Tak i >= 30
Nie
ryzyko_punkty += 1
ryzyko_punkty += 2
Więcej warunków?
Oblicz całkowite ryzyko
ryzyko <= 2?
Nie
Tak
ryzyko <= 5?
Tak
Nie
NISKIE RYZYKO
ŚREDNIE RYZYKO
WYSOKIE RYZYKO



2. Pętla while

Definition

Pętla while powtarza blok kodu tak długo, jak długo określony warunek pozostaje prawdziwy. Używana, gdy nie wiemy z góry, ile iteracji będzie potrzebnych (np. algorytm zbieżności, szukanie wartości, walidacja wejścia użytkownika).

Wprowadzenie

Pętla `while` jest uniwersalnym narzędziem do iteracji. W Data Science wykorzystujemy ją przede wszystkim w algorytmach optymalizacyjnych (gradient descent), szukaniu wartości granicznych, oraz w sytuacjach, gdzie liczba iteracji zależy od osiągnięcia określonego kryterium (konwergencji, dokładności).

Przykład 1: Podstawowa pętla while - Licznik

```
# Prosta pętla while: odliczanie
licznik = 5

print("Odliczanie:")
while licznik > 0:
    print(f"  {licznik}")
    licznik -= 1  # Dekrementacja: licznik = licznik - 1

print("Start!")

# Przykład z sumowaniem: suma liczb od 1 do n
n = 10
suma = 0
i = 1
```

```
while i <= n:
    suma += i  # suma = suma + i
    i += 1

print(f"\nSuma liczb od 1 do {n}: {suma}")
# Wzór sprawdzający: n * (n+1) / 2 = 10 * 11 / 2 = 55
```

Przykład 2: Pętla while z break - Symulacja zbieżności algorytmu

```
# Symulacja algorytmu gradient descent (uproszczona)
# Cel: znaleźć minimum funkcji poprzez iteracyjną poprawę

# Parametry początkowe
wartosc = 100.0  # Wartość startowa
krok = 0.1       # Szybkość uczenia (learning rate)
dokladnosc = 0.01  # Próg zbieżności
max_iteracji = 1000
iteracja = 0

print("=== SYMULACJA ALGORYTMU OPTYMALIZACYJNEGO ===")
print(f"Wartość początkowa: {wartosc:.4f}")
print(f"Próg dokładności: {dokladnosc}")

# Symulacja: każda iteracja zmniejsza wartość o 10%
while True:  # Pętla nieskończona z warunkiem wyjścia
    iteracja += 1

    # Oblicz nową wartość (symulacja kroku optymalizacji)
    nowa_wartosc = wartosc * 0.9  # Zmniejszenie o 10%
    zmiana = abs(wartosc - nowa_wartosc)

    # Wyświetl postęp co 100 iteracji
    if iteracja % 100 == 0:
        print(f"Iteracja {iteracja}: wartość={nowa_wartosc:.6f}, zmiana=
{zmiana:.6f}")

    # Warunek zbieżności: czy zmiana jest wystarczająco mała?
    if zmiana < dokladnosc:
        print(f"\n✓ Zbieżność osiągnięta w iteracji {iteracja}")
        print(f"  Wartość końcowa: {nowa_wartosc:.6f}")
        break  # Wyjście z pętli

    # Zabezpieczenie przed nieskończoną pętlą
    if iteracja >= max_iteracji:
```

```
        print(f"\n⚠ Osiągnięto maksymalną liczbę iteracji ({max_iteracji})")
        print(f"  Wartość końcowa: {nowa_wartosc:.6f}")
        break

    # Aktualizacja wartości
    wartosc = nowa_wartosc

print(f"Całkowita liczba iteracji: {iteracja}")
```

> ⚠ **Warning**
>
> Zawsze upewnij się, że warunek pętli `while` w końcu stanie się fałszywy! W przeciwnym razie program zawiesi się w nieskończonej pętli. Używaj zabezpieczeń typu `max_iteracji`.

Przykład 3: Pętla while z continue - Walidacja danych wejściowych

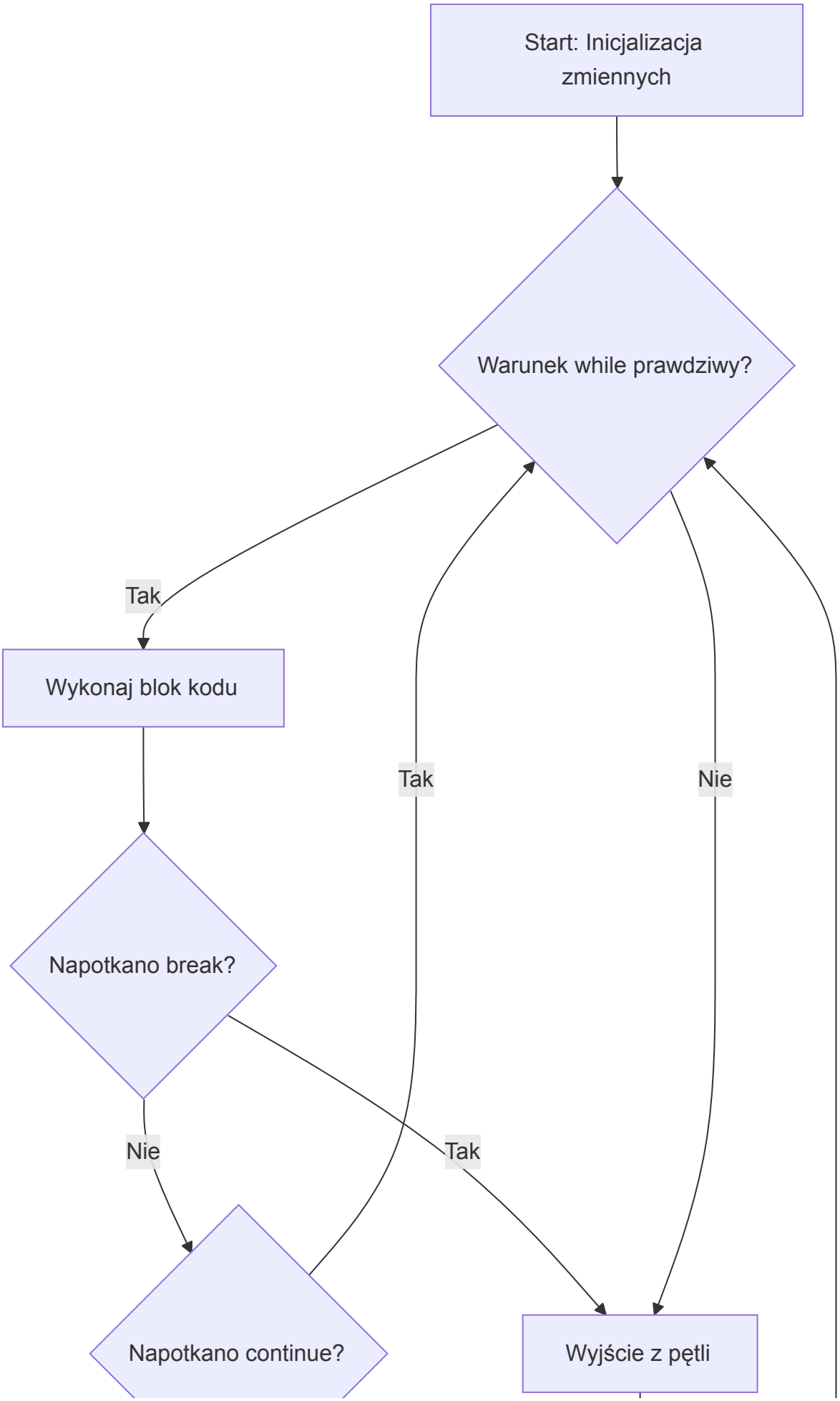
```
# Symulacja walidacji danych wejściowych dla systemu medycznego
# W prawdziwym systemie dane przychodzą z formularza/API

# Przykładowe dane - symulacja strumienia danych z czujników
# (w rzeczywistości użylibyśmy input() lub API)
dane_wejsciowe = [
    {"temperatura": 36.6, "puls": 72, "cisnienie_skurcz": 120},
    {"temperatura": 42.0, "puls": 180, "cisnienie_skurcz": 90},   # Błędne
dane
    {"temperatura": 37.2, "puls": 65, "cisnienie_skurcz": 115},
    {"temperatura": -10.0, "puls": 75, "cisnienie_skurcz": 125},  # Błędne
dane
    {"temperatura": 38.5, "puls": 88, "cisnienie_skurcz": 130},
]

print("=== WALIDACJA DANYCH MEDYCZNYCH ===\n")

dane_poprawne = []
dane_bledne = []
index = 0

while index < len(dane_wejsciowe):
    pomiar = dane_wejsciowe[index]

    # Walidacja zakresu temperatury (35-42°C)
    if pomiar["temperatura"] < 35 or pomiar["temperatura"] > 42:
        print(f"⚠ Pomiar #{index+1}: Temperatura poza zakresem:
```

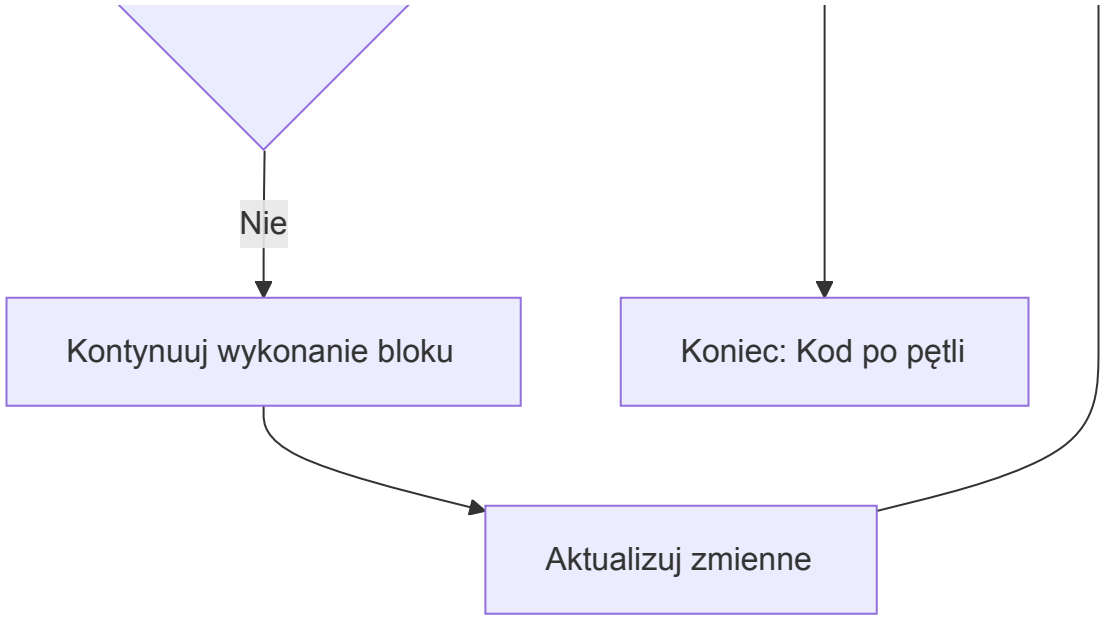
```
{pomiar['temperatura']}°C")
        dane_bledne.append(pomiar)
        index += 1
        continue  # Przeskocz do następnej iteracji

    # Walidacja pulsu (40-200 bpm)
    if pomiar["puls"] < 40 or pomiar["puls"] > 200:
        print(f"⚠ Pomiar #{index+1}: Puls poza zakresem: {pomiar['puls']}
bpm")
        dane_bledne.append(pomiar)
        index += 1
        continue

    # Walidacja ciśnienia skurczowego (80-200 mmHg)
    if pomiar["cisnienie_skurcz"] < 80 or pomiar["cisnienie_skurcz"] > 200:
        print(f"⚠ Pomiar #{index+1}: Ciśnienie poza zakresem:
{pomiar['cisnienie_skurcz']} mmHg")
        dane_bledne.append(pomiar)
        index += 1
        continue

    # Jeśli wszystkie walidacje przeszły pomyślnie
    print(f"✓ Pomiar #{index+1}: Dane poprawne - zapisano")
    dane_poprawne.append(pomiar)
    index += 1

# Podsumowanie
print(f"\n=== PODSUMOWANIE WALIDACJI ===")
print(f"Pomiary poprawne: {len(dane_poprawne)}")
print(f"Pomiary błędne: {len(dane_bledne)}")
print(f"Odsetek poprawnych:
{len(dane_poprawne)/len(dane_wejsciowe)*100:.1f}%")
```

Tip

Używaj `break` do natychmiastowego wyjścia z pętli, a `continue` do pominięcia pozostałej części aktualnej iteracji i przejścia do następnej. Zwiększa to czytelność kodu i pozwala unikać głęboko zagnieżdżonych `if`-ów.

Wizualizacja działania pętli while


Start: Inicjalizacja zmiennych
Warunek while prawdziwy?
Tak
Wykonaj blok kodu
Tak
Nie
Napotkano break?
Nie
Tak
Napotkano continue?
Wyjście z pętli



---

3. Pętla for i funkcja range()

Definition

Pętla for iteruje po elementach sekwencji (lista, string, range, etc.) lub innej struktury iterowalnej. Jest najczęściej używaną pętlą w Pythonie, szczególnie w Data Science do przetwarzania kolekcji danych, uczenia modeli, oraz obliczania statystyk na dużych zbiorach.

Wprowadzenie

Pętla `for` jest bardziej Pythonowa (pythonic) niż `while` w większości przypadków. W Data Science używamy jej do:

- Iteracji po wierszach/kolumnach danych
- Przetwarzania batch'y danych
- Trenowania modeli (iteracje po epokach)
- Obliczania metryk dla wielu modeli
- Feature engineering na wielu zmiennych

Funkcja `range()` generuje sekwencję liczb, co jest niezwykle użyteczne w połączeniu z pętlą `for`.

Przykład 1: Podstawowa pętla for - Iteracja po liście

```python
# Analiza wyników testów grupy uczniów
wyniki_testow = [85, 92, 78, 95, 88, 73, 90, 84, 96, 81]

print("=== ANALIZA WYNIKÓW TESTÓW ===\n")

# Iteracja po elementach listy
print("Wyniki poszczególnych uczniów:")
numer_ucznia = 1
for wynik in wyniki_testow:
    # Klasyfikacja wyniku
    if wynik >= 90:
        ocena = "Celujący"
    elif wynik >= 80:
        ocena = "Bardzo dobry"
    elif wynik >= 70:
        ocena = "Dobry"
    else:
        ocena = "Dostateczny"

    print(f"  Uczeń #{numer_ucznia}: {wynik} pkt - {ocena}")
    numer_ucznia += 1

# Obliczenia statystyczne
suma = 0
liczba = 0
najlepszy = wyniki_testow[0]
najgorszy = wyniki_testow[0]

for wynik in wyniki_testow:
    suma += wynik
    liczba += 1

    if wynik > najlepszy:
        najlepszy = wynik

    if wynik < najgorszy:
        najgorszy = wynik

srednia = suma / liczba

print(f"\n=== STATYSTYKI ===")
print(f"Średnia: {srednia:.2f}")
print(f"Najlepszy wynik: {najlepszy}")
print(f"Najgorszy wynik: {najgorszy}")
print(f"Rozstęp: {najlepszy - najgorszy}")
```

Note

W Pythonie możemy iterować bezpośrednio po elementach kolekcji (`for x in lista`), nie potrzebujemy indeksów jak w innych językach programowania.

Przykład 2: Funkcja range() - Generowanie sekwencji

```
# range() generuje sekwencję liczb
# range(stop) - od 0 do stop-1
# range(start, stop) - od start do stop-1
# range(start, stop, step) - od start do stop-1 co step

print("=== ZASTOSOWANIA FUNKCJI range() ===\n")

# Przykład 1: range(n) - od 0 do n-1
print("1. range(5) - liczby od 0 do 4:")
for i in range(5):
    print(f"  {i}", end=" ")
print("\n")

# Przykład 2: range(start, stop)
print("2. range(10, 15) - liczby od 10 do 14:")
for i in range(10, 15):
    print(f"  {i}", end=" ")
print("\n")

# Przykład 3: range(start, stop, step)
print("3. range(0, 20, 3) - co trzecia liczba od 0 do 19:")
for i in range(0, 20, 3):
    print(f"  {i}", end=" ")
print("\n")

# Przykład 4: Odwrotna iteracja
print("4. range(10, 0, -1) - odliczanie od 10 do 1:")
for i in range(10, 0, -1):
    print(f"  {i}", end=" ")
print("\n")

# ZASTOSOWANIE W DATA SCIENCE:
# Symulacja próbkowania danych co n-tą obserwację
print("\n=== PRÓBKOWANIE CO N-TĄ OBSERWACJĘ ===")
dane_id = list(range(1, 101))  # ID od 1 do 100
krok_probkowania = 10
```

```
print(f"Pełny dataset: {len(dane_id)} obserwacji")
print(f"Próbkowanie: co {krok_probkowania}-ta obserwacja")
print("Wybrane ID:", end=" ")

for i in range(0, len(dane_id), krok_probkowania):
    print(dane_id[i], end=" ")
print(f"\nLiczba próbek: {len(dane_id) // krok_probkowania}")
```

Przykład 3: Zaawansowana pętla for - Analiza szeregu czasowego

```
# Analiza tygodniowych pomiarów temperatury z czujników
# Symulacja danych z weather station

# Dane: temperatura średnia dzienna przez 14 dni (2 tygodnie)
temperatury = [
    15.2, 16.8, 18.3, 17.1, 14.5, 13.9, 15.7,  # Tydzień 1
    16.2, 18.9, 20.1, 21.3, 19.8, 18.4, 17.6   # Tydzień 2
]

dni = [
    "Pon", "Wt", "Śr", "Czw", "Pt", "Sob", "Nd",
    "Pon", "Wt", "Śr", "Czw", "Pt", "Sob", "Nd"
]

print("=== ANALIZA TEMPERATURY (14 DNI) ===\n")

# enumerate() - iteracja z indeksem
print("Dane pomiarowe:")
for indeks, temp in enumerate(temperatury):
    dzien_nazwa = dni[indeks]
    tydzien = (indeks // 7) + 1
    dzien_w_tygodniu = (indeks % 7) + 1

    print(f"  Tydzień {tydzien}, {dzien_nazwa} (dzień {dzien_w_tygodniu}):
{temp}°C")

# Analiza trendów: wykrywanie wzrostów i spadków
print("\n=== ANALIZA TRENDÓW ===")
wzrosty = 0
spadki = 0
bez_zmian = 0

for i in range(1, len(temperatury)):
    zmiana = temperatury[i] - temperatury[i-1]
```

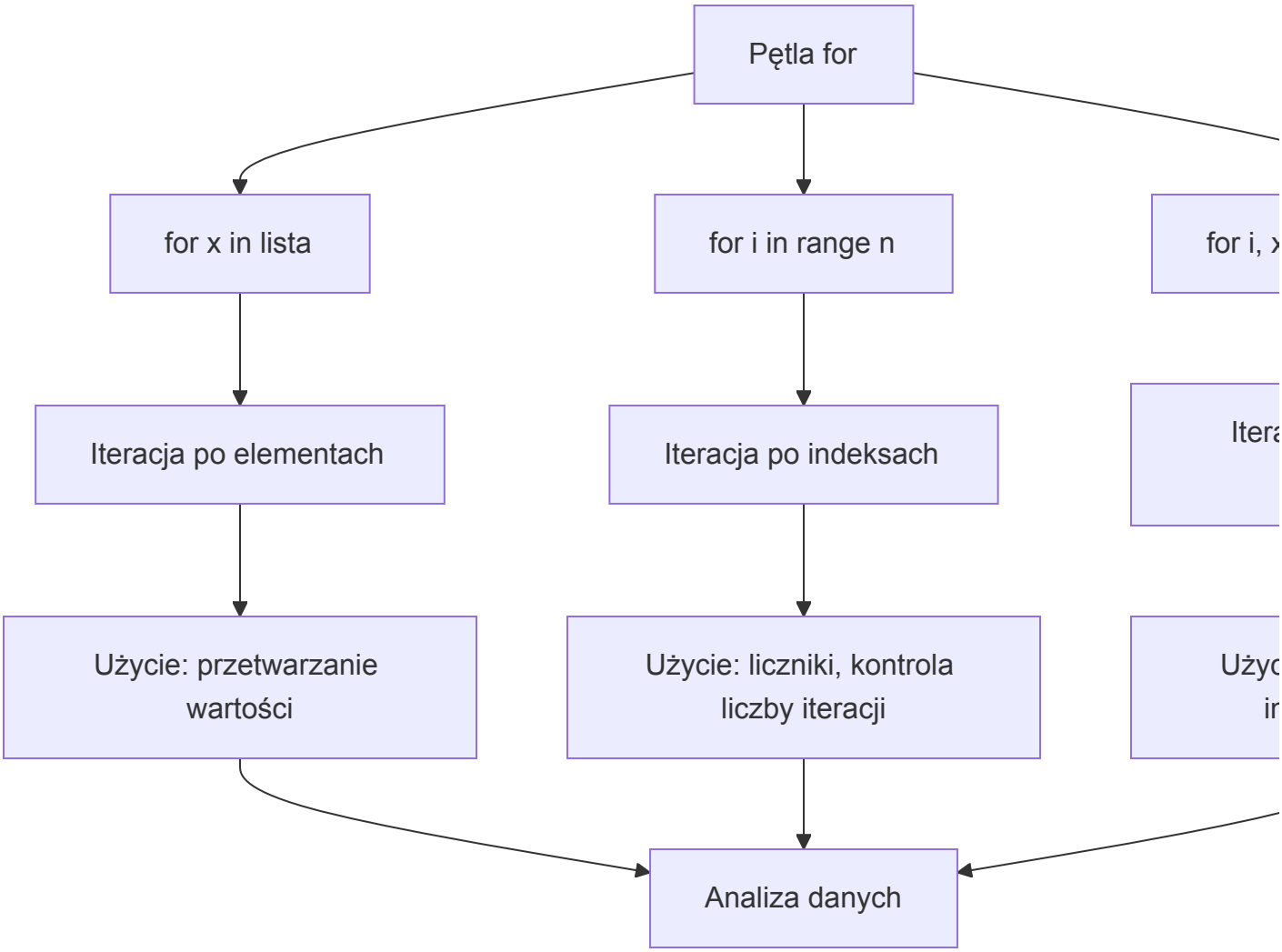
```python
    if zmiana > 0.5:  # Wzrost o więcej niż 0.5°C
        wzrosty += 1
        print(f"  ↑ Dzień {i+1}: wzrost o {zmiana:.1f}°C")
    elif zmiana < -0.5:  # Spadek o więcej niż 0.5°C
        spadki += 1
        print(f"  ↓ Dzień {i+1}: spadek o {zmiana:.1f}°C")
    else:
        bez_zmian += 1

# Statystyki tygodniowe
print("\n=== STATYSTYKI TYGODNIOWE ===")
for tydzien in range(2):
    start = tydzien * 7
    koniec = start + 7
    temperatury_tydzien = temperatury[start:koniec]

    suma_tydzien = 0
    for temp in temperatury_tydzien:
        suma_tydzien += temp

    srednia_tydzien = suma_tydzien / len(temperatury_tydzien)

    print(f"Tydzień {tydzien + 1}:")
    print(f"  Średnia: {srednia_tydzien:.2f}°C")
    print(f"  Minimum: {min(temperatury_tydzien):.1f}°C")
    print(f"  Maximum: {max(temperatury_tydzien):.1f}°C")

# Wykrywanie anomalii (outliers)
# Obliczenie średniej i odchylenia standardowego ręcznie
suma_wszystkich = 0
for temp in temperatury:
    suma_wszystkich += temp
srednia_calkowita = suma_wszystkich / len(temperatury)

suma_kwadratow_odchylen = 0
for temp in temperatury:
    odchylenie = temp - srednia_calkowita
    suma_kwadratow_odchylen += odchylenie ** 2

wariancja = suma_kwadratow_odchylen / len(temperatury)
odchylenie_std = wariancja ** 0.5

print(f"\n=== WYKRYWANIE ANOMALII ===")
print(f"Średnia: {srednia_calkowita:.2f}°C")
print(f"Odchylenie standardowe: {odchylenie_std:.2f}°C")
print(f"Próg anomalii: ±{2*odchylenie_std:.2f}°C od średniej\n")
```

```
for i in range(len(temperatury)):
    odchylenie_od_sredniej = abs(temperatury[i] - srednia_calkowita)

    if odchylenie_od_sredniej > 2 * odchylenie_std:
        print(f"  ⚠ Dzień {i+1}: {temperatury[i]}°C - ANOMALIA (odchylenie:
{odchylenie_od_sredniej:.2f}°C)")
```

✓ **Success**

Best practice: Używaj funkcji wbudowanych (`sum()`, `min()`, `max()`) zamiast pisać własne pętle, gdy to możliwe. Są szybsze i bardziej czytelne. Powyższy przykład pokazuje implementację "od zera" dla celów edukacyjnych.

Wizualizacja różnych zastosowań pętli for



4. Zagnieżdżone pętle

Definition

Zagnieżdżone pętle to pętle umieszczone wewnątrz innych pętli. W Data Science są niezbędne do pracy z danymi wielowymiarowymi (macierze, tensory), porównywania par obserwacji, oraz obliczania macierzy odległości.

Wprowadzenie

Zagnieżdżone pętle pozwalają na iterację wielowymiarową. Typowe zastosowania:

- Operacje na macierzach (przed użyciem NumPy)
- Porównywanie wszystkich par obiektów
- Generowanie kombinacji parametrów (grid search)
- Analiza korelacji między zmiennymi

Warning

Zagnieżdżone pętle mogą być bardzo kosztowne obliczeniowo! Dla `n` elementów w obu pętlach wykonujemy n^2 operacji. To jeden z głównych powodów używania bibliotek wektoryzowanych (NumPy, Pandas).

Przykład: Macierz odległości między punktami

```
# Obliczenie macierzy odległości między miastami
# Zastosowanie: logistics, routing, clustering

# Współrzędne miast (symulacja - uproszczony system współrzędnych)
miasta = ["Warszawa", "Kraków", "Gdańsk", "Wrocław", "Poznań"]
wspolrzedne = [
    (52.23, 21.01),  # Warszawa
    (50.06, 19.94),  # Kraków
    (54.35, 18.65),  # Gdańsk
    (51.11, 17.04),  # Wrocław
    (52.41, 16.93),  # Poznań
]

print("=== MACIERZ ODLEGŁOŚCI MIĘDZY MIASTAMI ===\n")

# Nagłówek tabeli
```

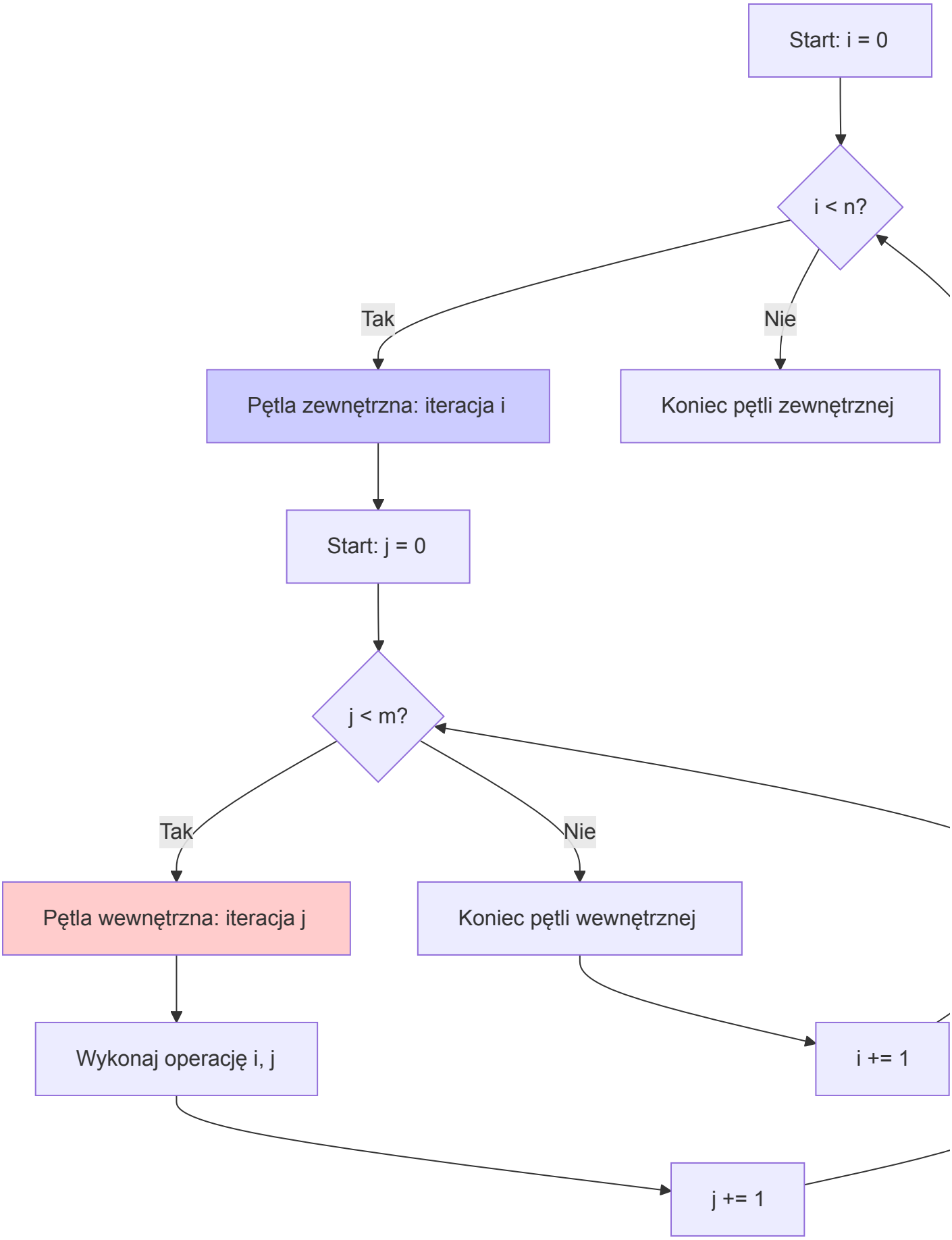
```
print("        ", end="")
for miasto in miasta:
    print(f"{miasto:>10}", end="")
print()

# Obliczenie i wyświetlenie macierzy
for i in range(len(miasta)):
    print(f"{miasta[i]:8}", end="")

    for j in range(len(miasta)):
        if i == j:
            # Odległość miasta od siebie samego = 0
            odleglosc = 0.0
        else:
            # Obliczenie odległości euklidesowej (uproszczone)
            lat1, lon1 = wspolrzedne[i]
            lat2, lon2 = wspolrzedne[j]

            # Różnice współrzędnych
            delta_lat = lat2 - lat1
            delta_lon = lon2 - lon1

            # Odległość euklidesowa (w jednostkach stopni - uproszczenie)
            odleglosc = ((delta_lat ** 2) + (delta_lon ** 2)) ** 0.5

            # Konwersja na kilometry (przybliżona: 1° ≈ 111 km)
            odleglosc = odleglosc * 111

        print(f"{odleglosc:>10.1f}", end="")

    print()  # Nowa linia po każdym wierszu

# Znalezienie pary miast najbliższych i najdalszych
print("\n=== ANALIZA PAR MIAST ===")

min_odleglosc = float('inf')  # Nieskończoność
max_odleglosc = 0
para_najblizsze = None
para_najdalsze = None

for i in range(len(miasta)):
    for j in range(i + 1, len(miasta)):  # j > i, aby nie liczyć dwa razy
tej samej pary
        lat1, lon1 = wspolrzedne[i]
        lat2, lon2 = wspolrzedne[j]
```

```
        delta_lat = lat2 - lat1
        delta_lon = lon2 - lon1
        odleglosc = ((delta_lat ** 2) + (delta_lon ** 2)) ** 0.5 * 111

        if odleglosc < min_odleglosc:
            min_odleglosc = odleglosc
            para_najblizsze = (miasta[i], miasta[j])

        if odleglosc > max_odleglosc:
            max_odleglosc = odleglosc
            para_najdalsze = (miasta[i], miasta[j])

print(f"Najbliższe miasta: {para_najblizsze[0]} - {para_najblizsze[1]}
({min_odleglosc:.1f} km)")
print(f"Najdalsze miasta: {para_najdalsze[0]} - {para_najdalsze[1]}
({max_odleglosc:.1f} km)")
```

Tip

Gdy iterujesz po parach unikalnych elementów, używaj `for i in range(len(lista))` dla pierwszej pętli i `for j in range(i+1, len(lista))` dla drugiej. To zapobiega liczeniu pary (A, B) i (B, A) jako osobnych.

Wizualizacja zagnieżdżonej pętli


Start: i = 0
i < n?
Tak
Nie
Pętla zewnętrzna: iteracja i
Koniec pętli zewnętrznej
Start: j = 0
j < m?
Tak
Nie
Pętla wewnętrzna: iteracja j
Koniec pętli wewnętrznej
Wykonaj operację i, j
i += 1
j += 1

5. List comprehensions - Pythonowy sposób na pętle

Definition

List comprehension to zwięzły, jednoliniowy sposób tworzenia list w Pythonie poprzez iterację z opcjonalnym filtrowaniem. Jest szybsza i bardziej czytelna niż tradycyjna pętla for z append().

Wprowadzenie

List comprehensions to idiom Pythona, który pozwala pisać kod bardziej zwięźle. W Data Science używamy ich ciągle do:

- Transformacji danych
- Filtrowania obserwacji
- Feature engineering
- Przygotowania danych do modeli

Składnia: `[wyrażenie for element in sekwencja if warunek]`

Przykład: List comprehensions w analizie danych

```
# Dane: wyniki badań laboratoryjnych (poziom glukozy u pacjentów)
glukozy_mg_dl = [95, 110, 78, 130, 88, 145, 92, 168, 102, 115, 85, 200, 98,
125]

print("=== LIST COMPREHENSIONS W ANALIZIE MEDYCZNEJ ===\n")

# Tradycyjna pętla for
wyniki_wysokie_tradycyjnie = []
for poziom in glukozy_mg_dl:
    if poziom >= 126:  # Próg dla diabetes
        wyniki_wysokie_tradycyjnie.append(poziom)

print("Tradycyjna pętla - wysokie poziomy:")
print(f"  {wyniki_wysokie_tradycyjnie}")

# List comprehension - to samo, krócej
wyniki_wysokie_lc = [poziom for poziom in glukozy_mg_dl if poziom >= 126]

print("\nList comprehension - wysokie poziomy:")
print(f"  {wyniki_wysokie_lc}")
```

```python
# Transformacja: konwersja mg/dL na mmol/L (1 mg/dL = 0.0555 mmol/L)
glukozy_mmol_l = [poziom * 0.0555 for poziom in glukozy_mg_dl]

print("\nKonwersja jednostek (mg/dL → mmol/L):")
for i in range(len(glukozy_mg_dl)):
    print(f"  {glukozy_mg_dl[i]} mg/dL = {glukozy_mmol_l[i]:.2f} mmol/L")

# Kategoryzacja z transformacją
kategorie = [
    "norma" if g < 100
    else "pre-diabetes" if g < 126
    else "diabetes"
    for g in glukozy_mg_dl
]

print("\nKategoryzacja pacjentów:")
for i in range(len(glukozy_mg_dl)):
    print(f"  Pacjent #{i+1}: {glukozy_mg_dl[i]} mg/dL → {kategorie[i]}")

# Statystyka: ile w każdej kategorii?
liczba_norma = sum([1 for k in kategorie if k == "norma"])
liczba_pre = sum([1 for k in kategorie if k == "pre-diabetes"])
liczba_diabetes = sum([1 for k in kategorie if k == "diabetes"])

print(f"\n=== ROZKŁAD KATEGORII ===")
print(f"Norma: {liczba_norma} ({liczba_norma/len(kategorie)*100:.1f}%)")
print(f"Pre-diabetes: {liczba_pre} ({liczba_pre/len(kategorie)*100:.1f}%)")
print(f"Diabetes: {liczba_diabetes}
({liczba_diabetes/len(kategorie)*100:.1f}%)")
```

✓ **Success**

Best practice: Używaj list comprehensions dla prostych transformacji i filtrów. Dla złożonej logiki używaj tradycyjnej pętli for - czytelność jest ważniejsza niż zwięzłość.

Porównanie wydajności: pętla vs list comprehension

```python
# Symulacja: która metoda jest szybsza?
print("\n=== PORÓWNANIE WYDAJNOŚCI (symulacja) ===")

# Duży dataset - 10000 pomiarów
import random
random.seed(42)
duzy_dataset = [random.randint(70, 200) for _ in range(10000)]
```

```python
# Metoda 1: Tradycyjna pętla
# (w prawdziwym porównaniu użylibyśmy timeit)
filtrowane_petla = []
for wartosc in duzy_dataset:
    if wartosc >= 126:
        filtrowane_petla.append(wartosc)

# Metoda 2: List comprehension
filtrowane_lc = [wartosc for wartosc in duzy_dataset if wartosc >= 126]

print(f"Dataset: {len(duzy_dataset)} pomiarów")
print(f"Wyfiltrowane (pętla): {len(filtrowane_petla)}")
print(f"Wyfiltrowane (LC): {len(filtrowane_lc)}")
print(f"Wyniki identyczne: {filtrowane_petla == filtrowane_lc}")
print("\n💡 List comprehension jest zazwyczaj 10-30% szybsza niż pętla for")
```

🌱 Aspekt środowiskowy

W tej lekcji zwracamy uwagę na:

1. Pisanie efektywnych pętli - mniej iteracji = mniej energii

```python
# ❌ NIEWYDAJNE: Zagnieżdżone pętle bez optymalizacji
n = 1000
suma = 0

for i in range(n):
    for j in range(n):
        if i == j:  # Wykonujemy n² porównań, a tylko n jest prawdziwych!
            suma += i

# ✅ WYDAJNE: Pojedyncza pętla
n = 1000
suma = 0

for i in range(n):
    suma += i  # Tylko n iteracji zamiast n²

print(f"Wynik (optymalizacja): {suma}")
print(f"Oszczędność: {n**2 - n} iteracji ({(1 - n/(n**2))*100:.1f}%
redukcja)")
```

2. Break i continue - wcześniejsze zakończenie = oszczędność zasobów

```
# Szukanie pierwszego wyniku spełniającego warunek
dane = list(range(1, 10001))

# ❌ NIEWYDAJNE: Przeszukanie całej listy nawet po znalezieniu
znaleziono = None
for x in dane:
    if x > 5000 and x % 7 == 0:
        znaleziono = x  # Znaleziono, ale pętla nadal leci dalej!

# ✅ WYDAJNE: Zakończenie po znalezieniu
znaleziono = None
iteracje = 0
for x in dane:
    iteracje += 1
    if x > 5000 and x % 7 == 0:
        znaleziono = x
        break  # Natychmiastowe wyjście

print(f"Znaleziono: {znaleziono}")
print(f"Przeszukano tylko {iteracje} z {len(dane)} elementów
({iteracje/len(dane)*100:.2f}%)")
print(f"Oszczędność: {len(dane) - iteracje} niepotrzebnych iteracji")
```

3. Range vs lista - lazy evaluation oszczędza pamięć

```
# ❌ NIEEFEKTYWNE: Tworzenie pełnej listy w pamięci
import sys
lista_duza = list(range(1000000))  # 1 milion elementów
pamiec_lista = sys.getsizeof(lista_duza)

# ✅ EFEKTYWNE: range() nie tworzy listy, generuje wartości on-demand
range_duzy = range(1000000)
pamiec_range = sys.getsizeof(range_duzy)

print(f"\n=== ZARZĄDZANIE PAMIĘCIĄ ===")
print(f"Lista 1 mln elementów: {pamiec_lista / 1024 / 1024:.2f} MB")
print(f"Range 1 mln elementów: {pamiec_range / 1024:.2f} KB")
print(f"Oszczędność pamięci: {pamiec_lista / pamiec_range:.0f}x")
print(f"Green IT: Używaj range() zamiast list() gdy nie potrzebujesz pełnej
listy")
```

4. Próbkowanie danych w pętlach - mniejszy dataset = szybsze przetwarzanie

```
# Podczas testowania kodu: używaj próbek, nie pełnych danych
pelny_dataset_rozmiar = 1000000
probka_rozmiar = 1000

# ❌ NIEWYDAJNE w fazie testowania: przetwarzanie pełnych danych
# for i in range(pelny_dataset_rozmiar):
#     # ... operacje testowe ...

# ✅ WYDAJNE w fazie testowania: próbkowanie
print(f"\n=== PRÓBKOWANIE PODCZAS TESTOWANIA ===")
print(f"Pełny dataset: {pelny_dataset_rozmiar} obserwacji")
print(f"Próbka testowa: {probka_rozmiar} obserwacji")
print(f"Przyspieszenie: {pelny_dataset_rozmiar / probka_rozmiar}x")
print(f"Oszczędność energii: ~{(1 -
probka_rozmiar/pelny_dataset_rozmiar)*100:.1f}%")
print(f"✓ Po przetestowaniu kodu uruchom na pełnych danych")
```

✓ **Success**

Green IT Best Practices:

- Unikaj zagnieżdżonych pętli gdy to możliwe (złożoność $O(n^2) \rightarrow O(n)$)
- Używaj `break` gdy znalazłeś wynik - nie iteruj dalej
- Preferuj `range()` nad `list(range())` dla dużych zakresów
- Testuj kod na próbkach danych, nie na pełnych datasetach
- Monitoruj złożoność czasową algorytmów

📊 Praca z Prawdziwymi Danymi

Dataset: Iris - klasyczny dataset w Machine Learning

W tej sekcji pokażemy, jak użyć pętli do analizy danych o kwiatach tęczówki (Iris dataset).

```
# Symulacja uproszczonego datasetu Iris (pierwsze 10 próbek)
# W rzeczywistości użylibyśmy pandas lub sklearn.datasets

iris_dane = [
```

```
    {"gatunek": "setosa", "dlugosc_platka": 1.4, "szerokosc_platka": 0.2},
    {"gatunek": "setosa", "dlugosc_platka": 1.4, "szerokosc_platka": 0.2},
    {"gatunek": "setosa", "dlugosc_platka": 1.3, "szerokosc_platka": 0.2},
    {"gatunek": "setosa", "dlugosc_platka": 1.5, "szerokosc_platka": 0.2},
    {"gatunek": "versicolor", "dlugosc_platka": 4.7, "szerokosc_platka":
1.4},
    {"gatunek": "versicolor", "dlugosc_platka": 4.5, "szerokosc_platka":
1.5},
    {"gatunek": "versicolor", "dlugosc_platka": 4.9, "szerokosc_platka":
1.5},
    {"gatunek": "virginica", "dlugosc_platka": 6.0, "szerokosc_platka":
2.5},
    {"gatunek": "virginica", "dlugosc_platka": 5.1, "szerokosc_platka":
1.9},
    {"gatunek": "virginica", "dlugosc_platka": 5.9, "szerokosc_platka":
2.1},
]

print("=== ANALIZA DATASETU IRIS ===\n")

# 1. Podstawowa eksploracja
print("Zawartość datasetu:")
for i, kwiat in enumerate(iris_dane):
    print(f"  Próbka #{i+1}: {kwiat['gatunek']:12} | "
          f"długość: {kwiat['dlugosc_platka']:.1f} cm | "
          f"szerokość: {kwiat['szerokosc_platka']:.1f} cm")

# 2. Grupowanie według gatunku
print("\n=== STATYSTYKI WEDŁUG GATUNKU ===")

gatunki_unikalne = []
for kwiat in iris_dane:
    if kwiat["gatunek"] not in gatunki_unikalne:
        gatunki_unikalne.append(kwiat["gatunek"])

for gatunek in gatunki_unikalne:
    # Filtrowanie danych dla danego gatunku
    dane_gatunek = []
    for kwiat in iris_dane:
        if kwiat["gatunek"] == gatunek:
            dane_gatunek.append(kwiat)

    # Obliczenia statystyczne
    suma_dlugosc = 0
    suma_szerokosc = 0
```

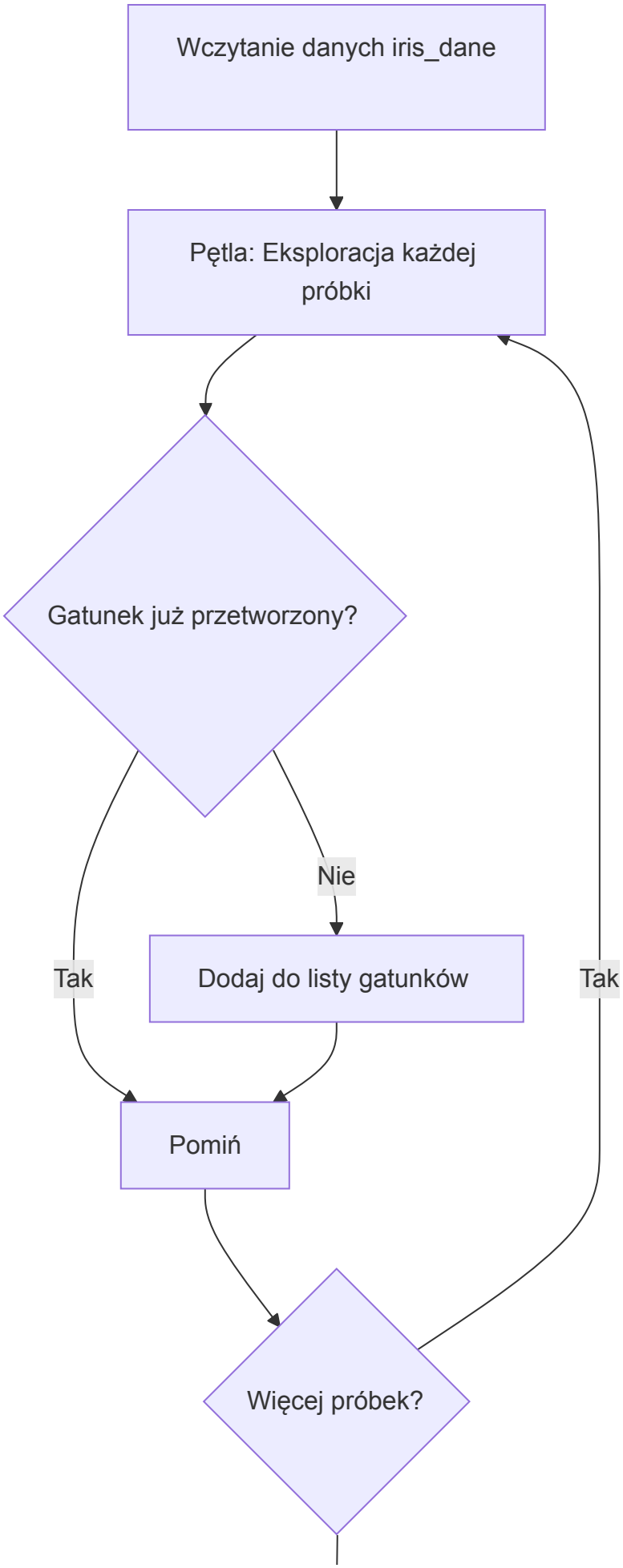
```
    for kwiat in dane_gatunek:
        suma_dlugosc += kwiat["dlugosc_platka"]
        suma_szerokosc += kwiat["szerokosc_platka"]

    liczba = len(dane_gatunek)
    srednia_dlugosc = suma_dlugosc / liczba
    srednia_szerokosc = suma_szerokosc / liczba

    print(f"\nGatunek: {gatunek}")
    print(f"  Liczba próbek: {liczba}")
    print(f"  Średnia długość płatka: {srednia_dlugosc:.2f} cm")
    print(f"  Średnia szerokość płatka: {srednia_szerokosc:.2f} cm")

# 3. Klasyfikacja na podstawie prostej reguły
print("\n=== PROSTA KLASYFIKACJA (RULE-BASED) ===")
print("Reguła: jeśli długość < 2.0 → setosa, jeśli 2.0-5.0 → versicolor,
jeśli > 5.0 → virginica\n")

poprawne = 0
bledne = 0

for kwiat in iris_dane:
    prawdziwy_gatunek = kwiat["gatunek"]
    dlugosc = kwiat["dlugosc_platka"]

    # Prosta reguła klasyfikacyjna
    if dlugosc < 2.0:
        przewidywany = "setosa"
    elif dlugosc < 5.0:
        przewidywany = "versicolor"
    else:
        przewidywany = "virginica"

    # Sprawdzenie poprawności
    if przewidywany == prawdziwy_gatunek:
        poprawne += 1
        status = "✓"
    else:
        bledne += 1
        status = "✗"

    print(f"  {status} Długość: {dlugosc:.1f} → Przewidywane:
{przewidywany:12} | Prawdziwe: {prawdziwy_gatunek}")

# Metryka accuracy
accuracy = poprawne / len(iris_dane) * 100
```

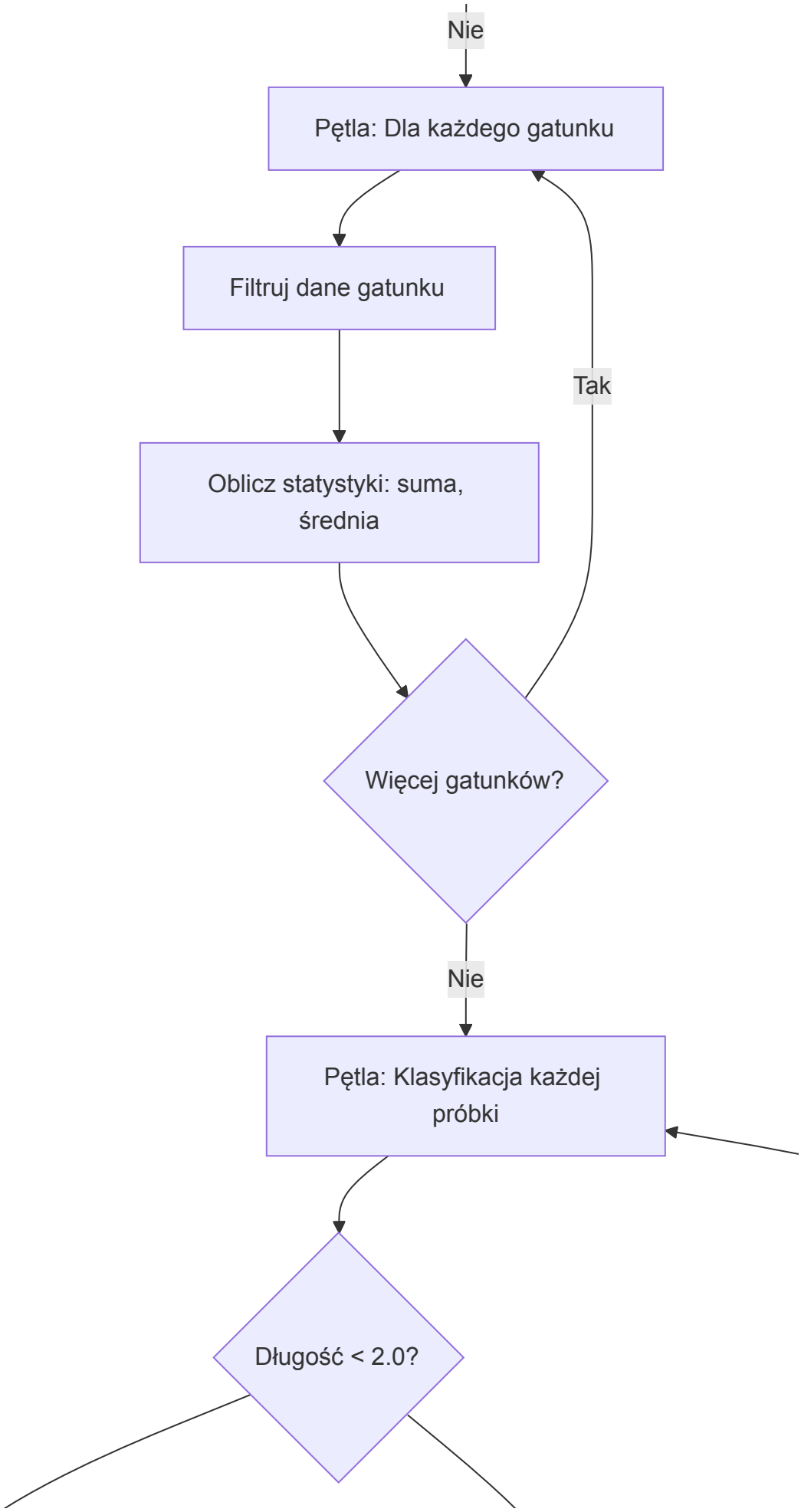
```
print(f"\n=== WYNIKI KLASYFIKACJI ===")
print(f"Poprawne: {poprawne}/{len(iris_dane)}")
print(f"Błędne: {bledne}/{len(iris_dane)}")
print(f"Accuracy: {accuracy:.1f}%")
```

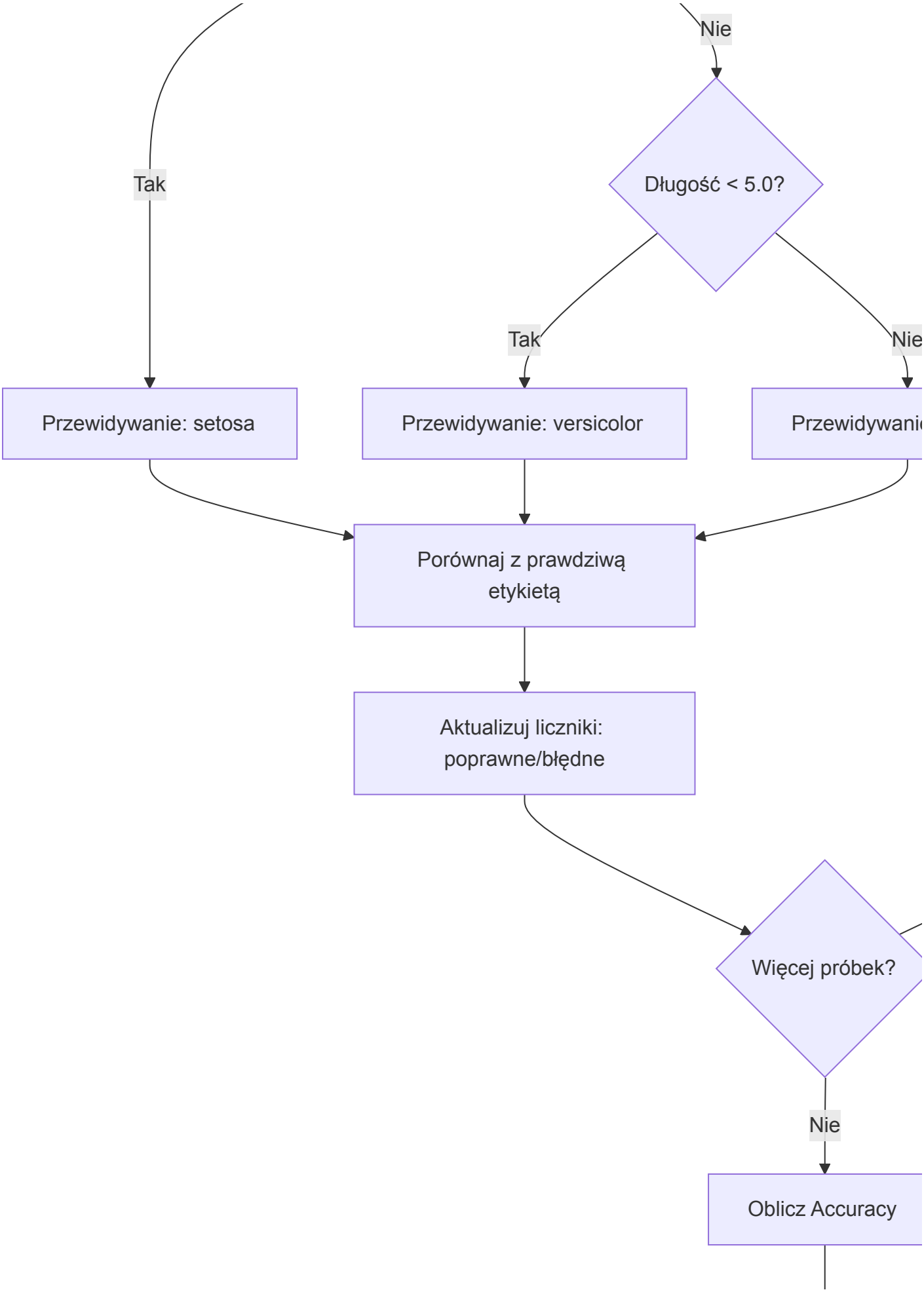
Note

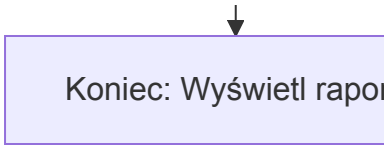
To uproszczona demonstracja. W prawdziwym projekcie Data Science użylibyśmy bibliotek Pandas (do przetwarzania danych) i Scikit-learn (do klasyfikacji). Jednak zrozumienie mechanizmów pętli pomaga lepiej rozumieć, co dzieje się "pod maską" tych bibliotek.

Wizualizacja przepływu analizy danych


Wczytanie danych iris_dane
Pętla: Eksploracja każdej próbki
Gatunek już przetworzony?
Nie
Tak
Dodaj do listy gatunków
Tak
Pomiń
Więcej próbek?


Nie
Pętla: Dla każdego gatunku
Filtruj dane gatunku
Tak
Oblicz statystyki: suma, średnia
Więcej gatunków?
Nie
Pętla: Klasyfikacja każdej próbki
Długość < 2.0?


Nie
Tak
Długość < 5.0?
Tak
Nie
Przewidywanie: setosa
Przewidywanie: versicolor
Przewidywani
Porównaj z prawdziwą etykietą
Aktualizuj liczniki: poprawne/błędne
Więcej próbek?
Nie
Oblicz Accuracy



🧪 Zadania do samodzielnej pracy

✏️ Zadania podstawowe (1-8)

1. ✏️ **Zadanie 1 – Klasyfikacja liczb parzystych i nieparzystych**

   Napisz program, który dla listy liczb `[12, 7, 23, 45, 8, 19, 34, 56, 91, 2]` wyświetli, które liczby są parzyste, a które nieparzyste. Użyj instrukcji `if/else` i operatora modulo `%`.

   Wymagania:

   - Iteruj po liście używając pętli `for`
   - Dla każdej liczby sprawdź: `if liczba % 2 == 0`
   - Wyświetl wynik w formacie: "12 - parzysta", "7 - nieparzysta"

   Oczekiwany rezultat:

   - Lista 10 linii z klasyfikacją każdej liczby

   (proste)
2. ✏️ **Zadanie 2 – Suma liczb dodatnich**

   Masz listę temperatury: `[-5, 3, -2, 8, 12, -1, 6, 0, 15, -3]`. Oblicz sumę tylko dodatnich wartości używając pętli `for` i instrukcji warunkowej.

   Wymagania:

   - Użyj zmiennej `suma = 0`
   - W pętli sprawdzaj `if temp > 0:`
   - Dodaj tylko dodatnie wartości do sumy

   Oczekiwany rezultat:

   - Wyświetlenie sumy dodatnich temperatur
   - Liczba dodatnich pomiarów

   (proste)
3. ✏️ **Zadanie 3 – Odliczanie z krokiem**

Napisz program, który wyświetli odliczanie od 20 do 0 z krokiem -2 (20, 18, 16, ..., 2, 0). Użyj pętli `for` i funkcji `range()`.

Wymagania:

- `for i in range(20, -1, -2):`
- Wyświetl każdą liczbę

Oczekiwany rezultat:

- Lista liczb: 20, 18, 16, 14, 12, 10, 8, 6, 4, 2, 0

(proste)

4. ✏️ **Zadanie 4 – Znajdź maksimum ręcznie**

Masz listę wyników testów: `[78, 92, 65, 88, 95, 71, 85]`. Znajdź maksymalny wynik używając pętli `for` (bez funkcji `max()`).

Wymagania:

- Inicjuj `maksimum = wyniki[0]`
- W pętli porównuj: `if wynik > maksimum:`
- Aktualizuj maksimum gdy znajdziesz większą wartość

Oczekiwany rezultat:

- Maksymalny wynik: 95

(proste)

5. ✏️ **Zadanie 5 – Suma liczb w zakresie**

Oblicz sumę wszystkich liczb od 1 do 50 używając pętli `while`.

Wymagania:

- Zmienne: `suma = 0`, `i = 1`
- `while i <= 50:`
- Dodaj `i` do sumy i zwiększ `i += 1`
- Sprawdź wynik wzorem: `n*(n+1)/2 = 50*51/2 = 1275`

Oczekiwany rezultat:

- Suma: 1275

(proste)

6. ✏️ **Zadanie 6 – Filtrowanie listy z warunkiem**

   Masz listę temperatur ciała: `[36.5, 37.8, 36.2, 38.5, 36.9, 39.2, 37.0]`. Wyświetl tylko te pomiary, które wskazują gorączkę (temperatura >= 38.0°C).

   Wymagania:

   - Pętla `for` po liście
   - Instrukcja `if temperatura >= 38.0:`
   - Wyświetl temperaturę z ostrzeżeniem

   Oczekiwany rezultat:

   - Lista temperatur >= 38.0 z komunikatem "Gorączka"

   (proste)

7. ✏️ **Zadanie 7 – Tabliczka mnożenia**

   Wyświetl tabliczkę mnożenia dla liczby 7 (od 7×1 do 7×10) używając pętli `for` i `range()`.

   Wymagania:

   - `for i in range(1, 11):`
   - Oblicz `wynik = 7 * i`
   - Wyświetl: "7 × 1 = 7"

   Oczekiwany rezultat:

   - 10 linii z wynikami mnożenia

   (proste)

8. ✏️ **Zadanie 8 – Konwersja listy używając list comprehension**

   Masz listę wzrostów w centymetrach: `[165, 178, 182, 159, 171, 190]`. Przekonwertuj je na metry używając list comprehension.

   Wymagania:

   - `wzrosty_metry = [wzrost / 100 for wzrost in wzrosty_cm]`
   - Wyświetl obie listy

   Oczekiwany rezultat:

   - Lista wzrostów w metrach: [1.65, 1.78, 1.82, 1.59, 1.71, 1.90]

   (proste)

✏️ Zadania średnie (9-12)

9. ✏️ **Zadanie 9 – Kategorie BMI dla listy pacjentów**

   Masz dane pacjentów: `wagi = [70, 85, 62, 95, 78]` (kg) i `wzrosty = [1.75, 1.80, 1.65, 1.78, 1.72]` (m). Oblicz BMI dla każdego i sklasyfikuj (niedowaga < 18.5, norma 18.5-24.9, nadwaga >= 25).

   Dataset:

   - Listy równoległe: wagi i wzrosty

   Wymagania:

   - Użyj pętli `for i in range(len(wagi)):`
   - Oblicz `bmi = wagi[i] / (wzrosty[i] ** 2)`
   - Sklasyfikuj używając `if/elif/else`
   - Wyświetl wyniki dla każdego pacjenta

   Oczekiwany rezultat:

   - 5 linii z BMI i kategorią dla każdego pacjenta

   (średnie)

10. ✏️ **Zadanie 10 – Znajdź wszystkie liczby pierwsze do 100**

    Napisz program, który znajdzie wszystkie liczby pierwsze od 2 do 100 używając zagnieżdżonych pętli.

    Wymagania:

    - Pętla zewnętrzna: `for n in range(2, 101):`
    - Pętla wewnętrzna: sprawdź dzielniki od 2 do √n
    - Zmienna `czy_pierwsza = True`
    - Jeśli znajdziesz dzielnik, ustaw `czy_pierwsza = False` i użyj `break`
    - Jeśli `czy_pierwsza == True`, dodaj do listy

    Oczekiwany rezultat:

    - Lista liczb pierwszych: [2, 3, 5, 7, 11, 13, ..., 97]
    - Liczba znalezionych: 25

    (średnie)

11. ✏️ **Zadanie 11 – Analiza trendów wzrostowych/spadkowych**

Masz dane sprzedaży miesięcznej: `[1200, 1350, 1290, 1480, 1520, 1510, 1680, 1720, 1700, 1890, 1950, 2100]`. Zidentyfikuj miesiące, w których sprzedaż wzrosła w porównaniu do poprzedniego miesiąca.

Dataset:

- Lista 12 wartości (sprzedaż miesięczna w PLN)

Wymagania:

- Pętla `for i in range(1, len(sprzedaz)):`
- Porównaj `sprzedaz[i]` z `sprzedaz[i-1]`
- Oblicz zmianę procentową
- Wyświetl miesiące ze wzrostem

Oczekiwany rezultat:

- Lista miesięcy ze wzrostem i procentowy przyrost

(średnie)

12. ✏️ **Zadanie 12 – Walidacja numeru PESEL (uproszczona)**

Sprawdź poprawność sum kontrolnych dla listy numerów PESEL (symulacja): `["92041812345", "85010154321", "78121698765"]`. Użyj pętli do iteracji po cyfrach.

Dataset:

- Lista 3 numerów PESEL (string)

Wymagania:

- Dla każdego PESEL iteruj po cyfrach
- Sprawdź długość (11 cyfr)
- Sprawdź czy wszystkie znaki to cyfry
- Oznacz jako "poprawny format" lub "błędny format"

Oczekiwany rezultat:

- Raport walidacji dla każdego numeru

(średnie)

🧠 Zadania wyzwanie (13-20)

13. 🧠 **Zadanie 13 – Symulacja algorytmu K-Nearest Neighbors (1D)**

Masz dane treningowe: `dane_treningowe = [(1.5, "A"), (2.3, "A"), (3.8, "B"), (4.2, "B"), (5.1, "B")]` (wartość, klasa). Dla nowego punktu `x_nowy = 3.5` znajdź 3 najbliższych sąsiadów i przewiduj klasę (najczęstsza klasa wśród sąsiadów).

Dataset:

- Lista krotek (wartość, etykieta)
- Nowy punkt: 3.5

Wymagania:

- Oblicz odległości: `abs(x_nowy - punkt)`
- Posortuj według odległości (ręcznie lub używając `sorted()`)
- Wybierz k=3 najbliższych
- Policz wystąpienia klas
- Przewiduj najczęstszą klasę

Oczekiwany rezultat:

- 3 najbliższe punkty z odległościami
- Przewidywana klasa dla x=3.5

(challenge)

14. 🧠 **Zadanie 14 – Generowanie macierzy korelacji (uproszczona)**

Masz dwie zmienne: `wzrost = [165, 170, 175, 180, 185]` i `waga = [60, 65, 70, 75, 80]`. Oblicz korelację Pearsona ręcznie używając pętli.

Dataset:

- Dwie listy równoległe

Wymagania:

- Oblicz średnie obu zmiennych
- Oblicz sumy: $\Sigma((x_i - \bar{x})(y_i - \bar{y}))$, $\Sigma(x_i - \bar{x})^2$, $\Sigma(y_i - \bar{y})^2$
- Korelacja $r = \Sigma((x_i - \bar{x})(y_i - \bar{y})) / \sqrt{(\Sigma(x_i - \bar{x})^2 \times \Sigma(y_i - \bar{y})^2)}$
- Użyj zagnieżdżonych obliczeń w pętli

Oczekiwany rezultat:

- Współczynnik korelacji r (oczekiwany: bliski 1.0 - silna korelacja dodatnia)

(challenge)

15. 🧠 **Zadanie 15 – Detekcja outliers metodą IQR**

    Dataset wyników egzaminów: `[78, 82, 85, 88, 90, 92, 95, 97, 99, 45, 101, 103, 105, 180]`. Wykryj outliers (wartości odstające) używając metody IQR (Interquartile Range).

    Dataset:

    - Lista wyników (zawiera outliers: 45 i 180)

    Wymagania:

    - Posortuj dane
    - Oblicz Q1 (25 percentyl) i Q3 (75 percentyl) ręcznie
    - IQR = Q3 - Q1
    - Outliers: wartości < Q1 - 1.5×IQR lub > Q3 + 1.5×IQR
    - Zidentyfikuj i wyświetl outliers

    Oczekiwany rezultat:

    - Q1, Q3, IQR
    - Lista outliers: [45, 180]

    (challenge)

16. 🧠 **Zadanie 16 – Symulacja Monte Carlo - Aproksymacja π**

    Użyj symulacji Monte Carlo do aproksymacji wartości π. Generuj losowe punkty w kwadracie [0,1]×[0,1] i sprawdzaj, ile z nich pada w ćwierci koła o promieniu 1.

    Wymagania:

    - Użyj modułu `random`
    - Wygeneruj 10000 losowych punktów (x, y)
    - Sprawdź warunek: `x² + y² <= 1` (punkt w kole)
    - π ≈ 4 × (punkty_w_kole / wszystkie_punkty)
    - Użyj pętli `for` do generowania i sprawdzania

    Oczekiwany rezultat:

    - Aproksymacja π (oczekiwana: ~3.14)
    - Błąd względny w procentach

    (challenge)

17. 🧠 **Zadanie 17 – Algorytm Euklidesa (NWD) z licznikiem iteracji**

Zaimplementuj algorytm Euklidesa do znajdowania największego wspólnego dzielnika (NWD) dwóch liczb: 252 i 105. Policz liczbę iteracji.

Wymagania:

- Użyj pętli `while`
- Algorytm: dopóki b ≠ 0: temp = b, b = a % b, a = temp
- Zlicz iteracje
- Wyświetl każdy krok

Oczekiwany rezultat:

- NWD(252, 105) = 21
- Liczba iteracji: 4

(challenge)

18. 🧠 **Zadanie 18 – Histogram ręczny (binning)**

Masz dane: `[12, 15, 18, 22, 25, 28, 31, 35, 38, 42, 45, 48, 52, 55, 58]`. Utwórz histogram z przedziałami (bins): [10-20), [20-30), [30-40), [40-50), [50-60). Policz ile wartości pada w każdy bin.

Dataset:

- Lista 15 wartości

Wymagania:

- Zdefiniuj granice binów: `[10, 20, 30, 40, 50, 60]`
- Dla każdej wartości znajdź odpowiedni bin używając pętli
- Zwiększ licznik dla tego binu
- Wyświetl histogram tekstowy (np. gwiazdki: *** dla 3 wartości)

Oczekiwany rezultat:

- Histogram pokazujący rozkład danych w przedziałach

(challenge)

19. 🧠 **Zadanie 19 – Macierz przejść (Transition Matrix)**

Masz sekwencję stanów systemu: `["A", "B", "A", "A", "C", "B", "C", "A", "B", "B"]`. Oblicz macierz przejść: ile razy stan X przechodzi w stan Y.

Dataset:

- Lista stanów (sekwencja)

Wymagania:

- Zidentyfikuj unikalne stany
- Utwórz strukturę do przechowywania przejść (np. słownik słowników)
- Pętla: dla i od 0 do len-2: zlicz przejście z `stany[i]` do `stany[i+1]`
- Wyświetl macierz przejść w formacie tabelarycznym

Oczekiwany rezultat:

- Macierz przejść (np. A→A: 1, A→B: 1, A→C: 1, B→A: 2, etc.)

(challenge)

20. 🧠 **Zadanie 20 – Implementacja Min-Max Normalizacji**

Masz dane: `wartosci = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]`. Znormalizuj je do zakresu [0, 1] używając Min-Max scaling: `x_norm = (x - min) / (max - min)`.

Dataset:

- Lista wartości

Wymagania:

- Znajdź minimum i maksimum używając pętli (bez `min()` / `max()`)
- Dla każdej wartości oblicz x_norm
- Zapisz znormalizowane wartości do nowej listy
- Sprawdź: minimum powinno być 0.0, maksimum 1.0

Oczekiwany rezultat:

- Lista znormalizowanych wartości [0.0, 0.111, 0.222, ..., 1.0]
- Weryfikacja zakresu [0, 1]

(challenge)

📚 Podsumowanie

W tej lekcji poznałeś:

- **Instrukcje warunkowe** (`if/elif/else`) - podstawa logiki decyzyjnej w programowaniu i Data Science

    - Klasyfikacja danych na podstawie warunków
    - Filtrowanie obserwacji
    - Implementacja reguł biznesowych
- **Pętlę while** - iteracja z warunkiem
    - Algorytmy zbieżności (gradient descent, optymalizacja)
    - Walidacja danych wejściowych
    - Kontrola przepływu z `break` i `continue`
- **Pętlę for** - najpopularniejsza pętla w Pythonie
    - Iteracja po kolekcjach (listy, stringi, range)
    - Funkcja `range()` - generowanie sekwencji liczb
    - `enumerate()` - iteracja z indeksem
    - Zagnieżdżone pętle - operacje wielowymiarowe
- **List comprehensions** - pythonowy idiom
    - Zwięzłe tworzenie list z transformacją
    - Filtrowanie danych w jednej linii
    - Wydajność vs czytelność
- **Aspekt środowiskowy (Green IT)**
    - Optymalizacja pętli → mniej iteracji = mniej energii
    - Używanie `break` i `continue` dla wcześniejszego zakończenia
    - `range()` vs `list()` - lazy evaluation oszczędza pamięć
    - Próbkowanie danych podczas testowania kodu

Połączenie z poprzednimi lekcjami:

- Instrukcje warunkowe operują na typach danych poznanych w Lekcji 4 (int, float, bool)
- Pętle iterują po kolekcjach z Lekcji 5 (listy, stringi) i Lekcji 6 (słowniki, zbiory, krotki)
- Operatory logiczne i porównania (Lekcja 4) są fundamentem warunków

Przygotowanie do następnych tematów:

- Funkcje (Lekcja 8) będą wykorzystywać pętle do implementacji logiki
- NumPy i Pandas (późniejsze moduły) oferują wektoryzację - alternatywę dla pętli
- Algorytmy Machine Learning intensywnie używają pętli (trenowanie modeli, ewaluacja)

Tip

Kluczowa zasada: Zawsze myśl o efektywności! Unikaj niepotrzebnych zagnieżdżeń, używaj odpowiednich struktur danych, testuj na próbkach. Kod, który wykonuje mniej operacji, jest nie tylko szybszy, ale też bardziej ekologiczny.

📖 Dodatkowe Zasoby

- Python Documentation - Control Flow - oficjalna dokumentacja Pythona
- Real Python - Python "for" Loops - szczegółowy tutorial o pętli for
- Real Python - Python "while" Loops - przewodnik po pętli while
- PEP 202 - List Comprehensions - specyfikacja list comprehensions
- Kaggle - Learn Python - interaktywny kurs z ćwiczeniami
- LeetCode - Python Practice - platforma z zadaniami algorytmicznymi
- Project Euler - matematyczne zadania programistyczne

Datasety do praktyki:

- Iris Dataset - klasyczny dataset ML
- Titanic Dataset - przeżywalność pasażerów Titanica
- Boston Housing - predykcja cen nieruchomości
- UCI ML Repository - baza danych dla Machine Learning